

# Tiny Inference: Low-Latency Language-to-Action Inference for Symbolic Robot Plans

Michael Mihaley

*High Performance Machine Learning, Spring 2026*

*Columbia University*

mjm2442@columbia.edu

**Abstract**—Large language models are increasingly used as high-level planners for robotics and embodied-AI systems, but interactive use requires low inference latency and predictable resource usage. This project studies a compact language-to-action workload that converts natural-language robot instructions into strict JSON action plans for a symbolic ROS 2/Gazebo pick-and-place demo. We compare a full-precision Hugging Face Transformers baseline against an optimized vLLM inference path using float16 and automatic prefix caching. On a single NVIDIA Tesla T4, the optimized path reduced median steady-state latency from 8598.23 ms to 2494.24 ms, a 3.45x speedup, while preserving the same task interface and using similar peak GPU memory.

**Index Terms**—inference optimization, large language models, vLLM, Transformers, ROS 2, latency benchmarking

## I. INTRODUCTION

Language models are useful high-level interfaces for robotics because they can translate natural-language goals into structured commands. However, even small instruction-tuned language models can be too slow for interactive systems when served with a straightforward single-request inference stack. In robotics-facing applications, latency is not just a convenience metric: slow planning delays feedback, weakens the sense of interaction, and can make the system harder to integrate into closed-loop workflows.

This project focuses on inference, not training. The workload is a language-to-action pipeline that prompts Qwen/Qwen2.5-3B-Instruct to produce a strict JSON plan containing symbolic actions such as `PICK`, `MOVE_TO`, and `PLACE`. The resulting plan is validated and sent to a small ROS 2/Gazebo symbolic interaction layer, where a lightweight symbolic arm moves boxes between tables. The optimization scope is intentionally narrow: preserve the same model, prompt, output schema, and downstream ROS interface, while reducing steady-state inference latency.

The final repository exposes only two tests. The baseline test uses Hugging Face Transformers in full precision on CUDA. The optimized test uses vLLM with float16 weights and automatic prefix caching. This restricted comparison makes the final result easy to reproduce and avoids mixing intermediate experiments with the submission interface.

## II. LITERATURE REVIEW

Transformer language models [2] have become the default architecture for instruction-following and planning-style

language interfaces. Modern open-weight instruction models make it practical to run multi-billion-parameter planners on commodity GPUs, but inference remains memory- and latency-sensitive because autoregressive decoding performs one token step at a time.

Several systems techniques address this bottleneck by reducing memory traffic and improving reuse during inference. FlashAttention reduces attention memory traffic by avoiding materialization of the full attention matrix [3]. In the same spirit, this project uses an inference engine with KV-cache management and automatic prefix caching so repeated chat-template prefixes do not need to be recomputed from scratch. These ideas matter even for small single-request demos because model loading, tokenization, prefill, decode, and cache behavior all affect whether an LLM planner feels usable.

Robotics work has also explored language models as planners or translators from natural language into robot-executable representations. Chen et al. study natural-to-robotic language translation with correctness guarantees [1]. SayCan connects language-model priors with robotic affordances [4], while Code as Policies uses language models to synthesize robot-control code [5]. Our project is smaller in scope: it does not claim real robot manipulation or closed-loop perception. Instead, it isolates the performance engineering problem of serving a fixed LLM planner quickly enough to drive a symbolic ROS 2 interaction.

## III. METHODOLOGY

### A. Workload and Data

The workload is a fixed instruction prompt stored in `etc/transform_prompt`. The model is asked to return a JSON object with a top-level plan array. Each element must be one of three symbolic actions: `PICK`, `MOVE_TO`, or `PLACE`. The ROS 2 package validates this schema and expands the plan into low-level symbolic commands for a Gazebo world containing `symbolic_arm`, `red_box`, `blue_box`, `round_table`, and `square_table`.

No external dataset is used. This is deliberate: the project measures serving latency for a representative prompt-driven planning request rather than task accuracy over a corpus. Correctness is defined by parseable JSON that passes the plan schema and can drive the symbolic demo.

## B. Model and Systems Stack

The model is Qwen/Qwen2.5-3B-Instruct, a 3B-parameter instruction-tuned causal language model. The baseline stack uses PyTorch and Hugging Face Transformers. The optimized stack uses vLLM. The downstream integration uses ROS 2 and Gazebo through `ros_gz_bridge`; the ROS package is intentionally symbolic only and does not depend on MoveIt, physical controllers, or trajectory action servers.

## C. Optimization Scope

The project optimizes inference only. Training, fine-tuning, quantization-aware training, and task-quality adaptation are out of scope. The final optimized configuration changes the serving backend and precision:

- Baseline: Transformers, CUDA, float32.
- Optimized: vLLM, CUDA, float16, automatic prefix caching, `max_num_seqs=1`, and `gpu_memory_utilization=0.8`.

The benchmark harness keeps the prompt, model name, output token limit, warmup count, and measured-run count fixed across the comparison. This makes the result a systems comparison rather than a prompt-engineering comparison.

## D. Profiling and Metrics

The benchmark records setup latency, input tokens, output tokens, average latency, p50 latency, p95 latency, tokens per second, peak GPU memory, and GPU utilization. GPU memory is sampled from PyTorch CUDA memory statistics and `nvidia-smi` when available. Runs can be logged to `Weights & Biases` as scalar histories and a `latency_reports` table.

Each reported run uses three warmup iterations and ten measured iterations. Warmup reduces the impact of one-time initialization in the steady-state latency numbers, while setup latency is still reported separately because startup cost matters for command-line and demo workflows.

# IV. EXPERIMENTAL RESULTS

## A. Experimental Setup

Measurements were collected on a single NVIDIA Tesla T4 GPU with 16 GB of memory, CUDA 12.4, Linux, and Python 3.10.15. The final before/after comparison came from the local W&B run `optimized-vs-base` on May 7, 2026. Both tests used the same model, prompt, warmup count, measured-run count, and `max_new_tokens=256`.

TABLE I  
BASELINE VS. OPTIMIZED INFERENCE RESULTS

| Metric          | Baseline    | Optimized   | Change         |
|-----------------|-------------|-------------|----------------|
| p50 latency     | 8598.23 ms  | 2494.24 ms  | 3.45x faster   |
| p95 latency     | 8691.40 ms  | 2923.45 ms  | 2.97x faster   |
| Throughput      | 16.63 tok/s | 28.66 tok/s | 1.72x higher   |
| Peak GPU memory | 11969 MB    | 12163 MB    | 1.6% higher    |
| Setup latency   | 8269.03 ms  | 34928.11 ms | slower startup |

Table I shows that the optimized path substantially reduces steady-state latency. Median latency fell from 8.60 s to 2.49

s, and p95 latency fell from 8.69 s to 2.92 s. Throughput increased from 16.63 tokens/s to 28.66 tokens/s. Peak memory was similar: the optimized path used 12163 MB versus 11969 MB for the baseline, an increase of about 194 MB.

## B. Analysis

The primary gain comes from replacing the general-purpose Transformers generation path with vLLM’s inference engine and enabling automatic prefix caching. In this workload, each request uses the same chat template and system prompt structure before the user instruction, so a portion of the prompt prefix can be reused across repeated requests. vLLM’s prefix caching avoids recomputing attention/KV state for matching prompt prefixes, reducing prefill overhead when the prompt server handles similar requests over time. Even though the benchmark measures one request at a time, the optimized path still benefits from vLLM’s lower-overhead decode loop and KV-cache management. Float16 further reduces memory bandwidth pressure relative to the full-precision baseline. The benchmarked prompt has 146 input tokens, so latency reflects both prompt prefill and autoregressive decoding.

The main tradeoff is startup latency. vLLM setup took 34.93 s compared with 8.27 s for the Transformers baseline. For a long-lived server, this startup cost is amortized, which matches the intended use of `prompt_inference_server.py`. For short one-off command-line invocations, startup latency can dominate the user experience. This is why the project includes both a single-inference script and a persistent prompt server.

The optimized path did not reduce peak GPU memory in the measured single-request setting. This is acceptable for the target T4 environment because both configurations fit within 16 GB, but it means the optimization should be described as a latency improvement rather than a memory reduction.

# V. DISCUSSION

The results show that serving infrastructure can matter as much as model choice for LLM planning workloads. Without changing the model or output schema, the optimized serving path made the symbolic planner feel much closer to an interactive component. This matters for the ROS 2 demo because the planner can be kept warm while the symbolic arm receives new prompt files and executes generated plans.

There are several limitations. First, the benchmark uses one fixed prompt rather than a broad instruction distribution. A larger evaluation should include diverse objects, locations, prompt lengths, and invalid-output cases. Second, the ROS layer is symbolic; it demonstrates integration and timing but not real grasp planning, perception, collision checking, or controller performance. Third, the optimized path was measured on a single Tesla T4. Results may differ on newer GPUs with better bfloat16 support, larger memory, and different vLLM kernels. Finally, the benchmark measures JSON-plan generation latency, not end-to-end task completion time in Gazebo.

Future work should extend the evaluation to multi-prompt sweeps, report confidence intervals, and separate prefill from decode time. On the systems side, batching and concurrent requests would make vLLM’s scheduling benefits more visible. On the robotics side, the symbolic arm could be replaced with a MoveIt-based manipulator after the inference benchmark is finalized, but that would introduce a different set of controller and planning bottlenecks.

## VI. CONCLUSION

This project optimized a small LLM-based language-to-action pipeline for inference latency. The final comparison keeps the public surface simple: a full-precision Transformers baseline and a vLLM float16 optimized path with prefix caching. On a Tesla T4, the optimized path achieved a 3.45x p50 latency speedup and a 1.72x throughput improvement while preserving the same model, prompt, JSON schema, and symbolic ROS 2 interaction. The main cost is slower engine startup, making the persistent prompt server the preferred deployment mode.

## REFERENCES

- [1] ZhenDong Chen et al., “An LLM-powered Natural-to-Robotic Language Translation Framework with Correctness Guarantees,” in *2025 International Joint Conference on Neural Networks (IJCNN)*, 2025.
- [2] A. Vaswani et al., “Attention is All You Need,” in *Advances in Neural Information Processing Systems*, 2017.
- [3] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Re, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *Advances in Neural Information Processing Systems*, 2022.
- [4] M. Ahn et al., “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances,” in *Proceedings of the Conference on Robot Learning*, 2022.
- [5] J. Liang et al., “Code as Policies: Language Model Programs for Embodied Control,” in *IEEE International Conference on Robotics and Automation*, 2023.

## APPENDIX A

### AI USE DISCLOSURE

The team used OpenAI Codex for code cleanup, README restructuring, and fixing grammatical errors in this LaTeX report.